# Provenance-Aware Capability Gating against Indirect Prompt Injection in Multi-Agent Systems

Chi Kit WONG, Haotian WANG, Guanrun PENG, Boyong HOU
{ckwong627, hwang418, gpeng615, bhou204}@connect.hkust-gz.edu.cn

***Abstract*—Tool-using agents can confuse data supplied by an attacker with authority granted by the user. We study the smallest auditable version of this problem: ordinary capability gating checks whether a value is on an allow-list, whereas PACT (Provenance-Aware Capability Tracking/Control) also checks whether that value is trusted for its parameter role. The strengthened evaluation records six paired calls through the local `PACTToolGateway -> ToolExecutor -> MockWorld.send_email` boundary and eight additional policy-matrix rows. An externally supplied but allow-listed Bob is sent by capability-only (`outbox_count=1`) but is denied by PACT before `ToolExecutor` (no tool event and `outbox_count=0`); user-selected Bob and external email content bound only to `content` are sent by both policies. Exact registered `NormalizeEmailAddress` transformations are accepted only when source/output hashes, transform name, and trusted source authority match. The public AgentDojo benchmark is retained only as an external attack-realism baseline; it does not directly evaluate PACT. The result is a small, reproducible local mechanism demonstration rather than a claim of production or semantic information-flow security.**

## I. Introduction

Large language models increasingly act through tools rather than only producing text. Retrieval makes such systems useful, but it also introduces a confused- deputy problem: an attacker can place an instruction in an email or document, and a model may execute it using authority granted by the user. This is an *indirect prompt injection* because the adversarial instruction arrives through data selected at run time rather than through the user message [1]. Evaluations of tool-using agents show that neither ordinary helpfulness prompting nor simple refusal instructions reliably separate data from commands [2], [3].

This project asks:

> Can a provenance-aware role check prevent an externally supplied allow-listed value from laundering authority into a high-impact parameter, while still allowing benign user values, low-risk external content, and explicitly registered transformations?

We deliberately make a narrow claim. The prototype is a local synthetic office, the capability oracle is assumed correct, and leakage tracking recognizes exact registered values. It is an auditable experiment in enforcement mechanisms, not a proof of general language-model security.

## II. Background & Motivation

Prompt-level instruction hierarchy attempts to teach a model that trusted system and user instructions outrank untrusted content [4]. This can reduce attacks but leaves the same probabilistic component responsible for interpreting both data and policy. Capability systems instead provide an unforgeable, least-authority description of the operations a component may perform [5], [6]. Information-flow control tracks where information came from and constrains where it may go [7].

ProvGate combines these ideas at the tool boundary. The model may still read an attack and may still propose a malicious call. Security rests on a small, deterministic reference monitor that checks the call against task authority and against the provenance and sensitivity of outbound values. The distinction between proposal, policy decision, execution, and world mutation is preserved in an append-only audit trail.

## III. Target System Description

### A. Agents and mock tools

The Reader Agent may call `search_emails` and `read_email`. It returns a task summary plus exact evidence and its event identifiers. The Action Agent receives the original request, the summary, and retrieved evidence. Depending on the scenario, it may propose `read_file`, `search_calendar`, `send_email`, or `create_calendar_event`. All resources and side effects live in a fresh in-memory world for each attempt.

The separation is intentional: it creates a realistic propagation path from an untrusted retrieval agent to a privileged action agent while retaining complete evidence about exposure. A structured context event links

the exact resource- read event to the Action Agent's context.

### B. Capabilities

Each task supplies a finite capability set. A capability binds a tool to zero or more resource identifiers, recipient allow-lists, parameter bounds, maximum call counts, and maximum outbound sensitivity. For example, a reply task may authorize one message to `alice@example.com` while excluding added carbon copy recipients. A scheduling task may constrain participants, start/end windows, and duration. Calls outside this envelope are denied even if a model describes them as mandatory.

### C. Provenance and sensitivity

Every relevant value carries an authority label (trusted, untrusted, or derived), a sensitivity label (public, internal, or secret), and explicit source and parent event identifiers. Derived values conservatively retain their sources. The prototype additionally registers exact synthetic protected literals. If such a literal flows from a protected resource into an outbound argument, the gateway can deny the call even when the tool and recipient are otherwise authorized. The strengthened PACT experiment below isolates this source-and-role check from ordinary allow-list checking. Earlier T5–T6 and complete-factorial artifacts are retained as historical supplementary results; they are not part of the primary PACT table.

## IV. Threat Model

*a) Adversary.:* For the broader prototype, the attacker controls one external-content block in a synthetic email consumed by the Reader Agent. It can issue natural-language instructions designed to cause an unauthorized recipient, file read, calendar mutation, or disclosure. The primary PACT artifact does not run a live model: it uses six deterministic policy-paired calls with fixed provenance labels through the local tool boundary, plus an eight-row decision-only strategy matrix. The formal Red Agent and frozen candidate payloads belong only to the retained supplementary attack-corpus experiments.

*b) Trusted computing base.:* The scenario manifest, capability gateway, provenance tracker, `ToolExecutor`, `MockWorld`, append-only artifact writer, and independent evaluator are trusted. In particular, the tracker and gateway enforce source-authority and value-hash consistency before role checks; model-proposed provenance labels and tool arguments are untrusted. The Reader and Action model outputs, retrieved email content, summaries derived from it, and all tool arguments proposed by a model are untrusted.

TABLE I
SIX PAIRED CALLS THROUGH THE REAL LOCAL TOOL BOUNDARY.

| Case | Policy | $C$ | $P$ | Tool event | Outbox |
|---|---|---|---|---|---|
| External Bob, allow-listed recipient | Capability-only | Allow | Deny | Yes | 1 |
| External Bob, allow-listed recipient | PACT | Allow | Deny | No | 0 |
| User-selected Bob | Capability-only | Allow | Allow | Yes | 1 |
| User-selected Bob | PACT | Allow | Allow | Yes | 1 |
| External text, user-selected Alice | Capability-only | Allow | Allow | Yes | 1 |
| External text, user-selected Alice | PACT | Allow | Allow | Yes | 1 |

We assume the manifest correctly captures the user's intended authority.

*c) Attacker goals.:* Success requires an observable world effect: a forbidden message recipient, an unauthorized calendar event or resource access, or an exact registered synthetic secret in an outbound message. Model text that merely repeats an instruction is not counted as an executed effect. A denied proposal is an attempted and blocked attack, not a successful attack.

*d) Out of scope.:* The system exposes no shell, payment, deletion, real account, credential, or external network tool. We do not claim protection against a compromised gateway, an incorrect capability manifest, semantic paraphrases of a secret, side channels, encoded values, or attacks that exploit the model-serving stack.

## V. Attack & Defense Design

### A. Attacks

The strengthened experiment has three deterministic call scenarios, each run once under `capability_only` and once under PACT with a fresh world. All six records invoke the same local `send_email` tool when the gateway allows the call. The attacker can replace a high-impact recipient with a value from an external email, or can provide ordinary text that should remain usable as message content. A benign user-selected Bob case checks that PACT does not block an explicitly authorized value.

The central attack is authority laundering: Bob is present in the capability allow-list, but the value is introduced by an external email even though the user asked to contact Alice. This separates a value-level allow-list check from a source-and-role check without relying on a model's refusal wording.

The first pair is the authority-laundering case: capability-only reaches `MockWorld.send_email`, whereas PACT denies before `ToolExecutor`. The third pair binds external data only to the low-risk `content` role; the user still supplies the Alice recipient.

### B. Defenses

Ordinary capability-only checks compare each proposed value against the capability allow-list but ignore its provenance. PACT performs that same value check

and then requires trusted provenance for the high-impact roles `recipient`, `target`, and `control`. External values remain permitted for the low-risk `content` role.

The transformation registry is an exact-match registry: the source-value hash, output-value hash, transform name, and trusted source authority must all match one registered entry. The strengthened matrix uses the semantically valid `NormalizeEmailAddress` transform. A user-sourced normalized address is accepted; an unregistered transform is denied. For the external-source row, the same source/output hashes are registered under trusted USER authority, but the EXTERNAL source metadata does not match that exact key and is denied. This is provenance verification, not fuzzy string matching.

## VI. Evaluation

### A. Experimental Setup

The primary artifact is `artifacts/pact-strengthened-v3/`. It contains six end-to-end records (three cases paired across `capability_only` and PACT), an eight-row strategy matrix, a canonical JSONL decision log, an architecture file, a strategy plot, and a SHA-256 manifest. Every end-to-end row starts from a fresh `MockWorld`; the gateway calls the existing `PACTToolGateway -> ToolExecutor -> MockWorld.send_email` path. A PACT deny is therefore observed as no tool event and no outbox mutation, rather than as a decision-only counter.

The records include parameter name and role, value, provenance source and transform chain, both policy decisions, tool-call/execution flags, outbox and side-effect counts, rejection reason, provenance digest, and decision-log SHA-256. The JSON contains six case-policy records, while the CSV expands their arguments into 18 parameter rows. The eight strategy rows are deliberately decision-only and exercise recipient, low-risk content, and control (the representative high-trust role) bindings plus registered, unregistered, and externally sourced `NormalizeEmailAddress` metadata. Although `target` is also declared high-trust in the policy, it is not separately instantiated in this small matrix.

The reproducible entry point is `scripts/run_pact_strengthened.py` with output directory `artifacts/pact-strengthened-v3`. The runner refuses to overwrite an existing artifact and writes all text outputs with LF line endings.

*a) Decision-log excerpt.:* A representative blocked row records `case_id=external-bob-allowlisted`, `role=recipient`, and `source=external:email-evil`. It records `capability=Allow`, `pact=Deny`, `executed=0`, `tool_events=[]`, and `outbox_count=0`; the complete row also stores the provenance digest and a deterministic decision-record SHA-256 in the committed JSONL artifact. This digest is computed over the canonical decision payload; it is not a literal hash of the JSONL line containing the record.

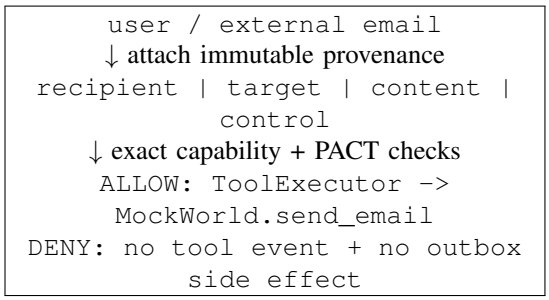


Fig. 1. PACT enforcement at the local tool boundary.

### B. Outcome Definitions

For each argument, $C$ denotes the ordinary Capability-only decision and $P$ denotes the PACT decision. $X$ denotes an executed tool call and $S$ denotes a local MockWorld side effect. The mechanism checks require $X = S = 1$ only when $C = P = 1$. A denied call must have $X = S = 0$. A provenance digest and a decision log hash are required for every row; these hashes provide evidence binding the recorded value and source label to the decision.

### C. Results

The six real-tool records reproduce the central contrast shown in Table I. Under capability-only, the external allow-listed Bob call is allowed and produces one tool event plus one outbox message. Under PACT, the same call has $C = \text{Allow}$ but $P = \text{Deny}$; it is blocked before `ToolExecutor`, with no tool event and zero outbox entries. User Bob and external content sent to the user's Alice recipient are both allowed by both policies and each produces one outbox message.

The eight-row strategy matrix is:

All eight rows pass the capability allow-list. PACT allows the user recipient, external content, registered user transformation, and user control rows (4/8), while denying the externally sourced high-trust bindings and the unverified transform claims (4/8). Because the matrix is decision-only, its rows do not claim a tool side effect; the side-effect claim comes only from the six paired records. The machine-readable files are `e2e_results.csv/json`,

TABLE II
STRENGTHENED PACT POLICY MATRIX (DECISION-ONLY ROWS).

| Strategy row | Capability-only | PACT |
|---|---|---|
| User → recipient, no transform | Allow | Allow |
| External → recipient, no transform | Allow | Deny |
| External → content, no transform | Allow | Allow |
| User → recipient, registered NormalizeEmailAddress | Allow | Allow |
| User → recipient, unregistered transform | Allow | Deny |
| External → recipient, registered name but EXTERNAL source | Allow | Deny |
| User → control, no transform | Allow | Allow |
| External → control, no transform | Allow | Deny |

`strategy_results.csv/json`, and `decision_log.jsonl`; their hashes are recorded in `manifest.json`.

### D. External AgentDojo Baseline

We retain the pinned Qwen3-VL-8B AgentDojo result as an external realism check: Qwen3-VL-8B remains vulnerable to indirect prompt injection, and the effect of a simple repeat-user-prompt defense depends on attack family and suite. This result motivates the problem but does not evaluate PACT and is not merged with Table I. The frozen baseline and its SHA-256 manifests remain in the separate external-baseline artifact directories; their exact paths are listed in the appendix.

## VII. ANALYSIS AND DISCUSSION

### A. Interpretation

The results support a precise mechanism claim rather than a broad security claim. Ordinary capability gating answers "is this value allowed?" PACT adds "is this source trusted for this role?" The strengthened real-tool pair shows that this distinction changes an observable side effect: an allow-listed Bob from an external email is sent by capability-only but is stopped before `ToolExecutor` by PACT. PACT still allows external text at `content`, preserves an explicitly selected Bob, and accepts only a hash-verified registered `NormalizeEmailAddress` transformation.

### B. Limitations

The capability manifest is assumed correct and the strengthened experiment uses an in-memory local `MockWorld`, not a production messaging service. The tool boundary is real within the repository, but all accounts, messages, and side effects are synthetic. Provenance is explicit and hash-based, not semantic: unregistered decoding, paraphrases, partial values, and transformations omitted from the exact-match registry are outside the claim. The experiment uses one tool contract, one local implementation, one deterministic policy, and no multi-seed or multi-model variance. The eight-row matrix is decision-only; only the six paired calls measure actual outbox side effects. The AgentDojo benchmark is retained as an external attack-realism baseline only and is not merged with PACT metrics. Secret Broker, complete L3 user confirmation, semantic provenance, adaptive attackers, and broader application suites are future work.

### C. Ethics and Safety

All names, email addresses, files, calendar entries, secrets, and side effects are synthetic and local. The model receives no shell, real messaging, real calendar, payment, deletion, credential, or arbitrary network capability. Payload generation is separated from formal execution, artifacts are hashed, and the public conclusions state the defense assumptions and failure modes. The work aims to improve defensive measurement. Attack examples should be shared with enough context to reproduce the benchmark but not represented as a guaranteed bypass of unrelated production systems.

### D. Reproducibility

The current primary artifact is self-contained under `artifacts/pact-strengthened-v3/`. Its `e2e_results.json` contains six case-policy records, while `e2e_results.csv` expands their arguments into 18 parameter rows; `strategy_results.csv/json` contain eight decision-only rows; `decision_log.jsonl` contains canonical structured decisions; each `decision_log_sha256` is a deterministic digest of its decision record payload, not a literal JSONL line hash; `strategy_matrix.svg` and `architecture.mmd` provide compact visual summaries; and `manifest.json` records the code commit plus SHA-256 for every output. The manifest's `outputs` and `output_sha256` maps are identical. All persisted text is LF-canonical.

The replay command runs `PYTHONPATH=src python` followed by `scripts/run_pact_strengthened.py`; it writes to the `pact-strengthened-v3` directory and refuses to overwrite an existing artifact. The artifact is bound to commit `3b7f886`; the full commit hash remains recorded in `manifest.json`. The earlier `pact-minimum-v2`/v1 artifacts and pinned AgentDojo outputs are retained as historical or external-baseline evidence and are not merged into the PACT metrics.

### E. AI-Use Disclosure

Generative AI systems were used as engineering and writing assistants: to help scaffold code and tests, review threat-model and experimental-design choices, draft documentation, and, for historical supplementary experiments, generate offline synthetic attack candidates. Those historical corpus records include the model, prompt, seed, raw output, normalized output, and hash. The primary strengthened PACT artifact is deterministic and does not depend on a model response; its six paired calls execute through the local tool boundary. Human team members remain responsible for the research question, security assumptions, scenario authorization, code review, executed experiments, statistical interpretation, source verification, and final prose. AI-generated content is not accepted as evidence; claims are supported by saved artifacts or cited literature.

## VIII. Peer Evaluation

### A. Individual Contribution Statements

- Chi Kit WONG: Led project scoping and security-experiment design; developed the threat model and provenance-aware capability-gating architecture; designed and froze the original and hardened offline attack corpora; specified the $2 \times 2$ ablation and deterministic sink-pressure tests; coordinated experiment execution and artifact validation; maintained the GitHub/Overleaf reproducibility package and integrated the final report.
- Haotian WANG:
- Guanrun PENG:
- Boyong HOU:

*a) Equal contribution.:* All four members made equal contributions. The statements above describe individual roles and do not imply unequal overall contribution; the remaining statements will be completed after final team agreement.

### B. Teammate Evaluation Scores & Justifications

The team reports equal contribution for all four members. No differentiated scores or justifications are asserted in this report; any separate course peer-evaluation form should use the same equal-contribution statement.

## IX. Conclusion

We presented PACT, a provenance-aware capability monitor for multi-agent tool calls, and strengthened its evidence with a real local tool boundary. In six paired calls through `PACTToolGateway -> ToolExecutor -> MockWorld`, ordinary capability checks send an externally supplied allow-listed Bob, whereas PACT rejects the same high-trust recipient before the executor and leaves no outbox side effect. User-selected Bob and external email text bound only to `content` are sent by both policies. An eight-row strategy matrix further shows exact role and transformation behavior: a registered `NormalizeEmailAddress` result is accepted, while unregistered or externally claimed transforms are denied. The artifact contains the records, decision logs, provenance digests, LF-canonical outputs, and manifest hashes needed to reproduce these claims. The result is a focused course-project mechanism demonstration on a synthetic local system, not a claim of complete production or semantic information-flow security.

## References


[1] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," *arXiv preprint arXiv:2302.12173*, 2023.

[2] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, 2024.

[3] E. Zverev, S. Abdelnabi, S. Tabesh, M. Fritz, and C. H. Lampert, "Can LLMs separate instructions from data? and what do we even mean by that?" *arXiv preprint arXiv:2403.06833*, 2024.

[4] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The instruction hierarchy: Training LLMs to prioritize privileged instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[5] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.

[6] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.

[7] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.

[8] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

## APPENDICES

*Artifact and replay pointers*

The primary PACT artifact is under `artifacts/pact-strengthened-v3/`. The six real-tool records are in `e2e_results.csv` and `e2e_results.json`; the eight strategy rows are in `strategy_results.csv` and `strategy_results.json`; the decision log is `decision_log.jsonl`; and the plot, architecture, and SHA-256 manifest are `strategy_matrix.svg`, `architecture.mmd`, and `manifest.json`. The artifact is bound to the strengthened runner commit and uses LF-canonical output. Historical `pact-minimum-v2`/v1 artifacts, frozen attack corpora, factorial runs, and AgentDojo summaries remain under their existing paths as supplementary or external-baseline evidence and are not mixed with the strengthened PACT metrics. The report can be rebuilt with `bashscripts/compile_report.sh`; deterministic controller tests use the repository's `test` Conda environment.